

CS214 Coding Assignment HW2

Hugo Wan, GitHub ID: HugoWan0504

Deadline: 4/27/2026

1 Introduction

The goal of this assignment is to implement a parallel exclusive prefix sum algorithm. For an input array A , the exclusive scan replaces each position with the sum of all previous elements, so $A[0]$ becomes 0 and $A[i]$ becomes $\sum_{j=0}^{i-1} A[j]$. Compared to the inclusive prefix sum discussed in lecture, the exclusive version shifts the accumulated result by one position and avoids including the current element in the sum. This operation is a fundamental building block in many parallel algorithms, such as sorting and graph processing.

In this assignment, I implemented an in-place blocked parallel scan in `scan.h`, while keeping the required function interface unchanged as specified in the instructions. The implementation focuses on achieving both correctness and good parallel performance under the given runtime constraints.

2 Algorithm

My implementation divides the input array into fixed-size blocks with a block size of 10000 elements. This choice provides a balance between parallelism and overhead, ensuring that each parallel task has enough work to be efficient. The total number of blocks is determined based on the input size, and each block is processed independently in parallel using `parallel_for`.

Inside each block, I perform a local sequential exclusive scan. During this process, each element is updated in place, and the total sum of the block is recorded in a separate `block_sum` array. This step allows the algorithm to compute partial prefix sums without requiring synchronization between blocks.

After all local scans are completed, I perform a sequential scan over the `block_sum` array. Since the number of blocks is much smaller than the total number of elements, this step adds only a small overhead. The result of this step gives each block a global offset, representing the sum of all elements in the preceding blocks.

Finally, another `parallel_for` is used to add the corresponding offset to every element within each block. This step combines the local scan results with the global offsets to produce the final exclusive prefix sum. Overall, the algorithm follows the scan-up and scan-down idea introduced in lecture: first compute partial sums in parallel, and then propagate the accumulated offsets back to obtain the correct global result.

```

#include <math.h>
#include <algorithm>

#include "parallel.h"
using namespace parlay;

/**
 * Inplace block scan DIRECTLY without filter
 */
template <typename T>
T scan(T *A, size_t n) {
    if (n == 0) return 0;

    const size_t BLOCK_SIZE = 10000;
    size_t num_blocks = (n + BLOCK_SIZE - 1) / BLOCK_SIZE;

    T *block_sum = new T[num_blocks];

    // scan-up idea: each block computes local exclusive scan and block total
    parallel_for(0, num_blocks, [&](size_t b) {
        size_t start = b * BLOCK_SIZE;
        size_t end = std::min(start + BLOCK_SIZE, n);

        T total = 0;
        for (size_t i = start; i < end; i++) {
            T tmp = A[i];
            A[i] = total;
            total += tmp;
        }

        block_sum[b] = total;
    });
}

```

Figure 1: First part of the implemented blocked exclusive scan in `scan.h`, including block creation and local block scans.

```

// scan the block sums sequentially because num_blocks is much smaller
T total = 0;
for (size_t b = 0; b < num_blocks; b++) {
    T tmp = block_sum[b];
    block_sum[b] = total;
    total += tmp;
}

// scan-down idea: add each block's offset back to its local scan result
parallel_for(0, num_blocks, [&](size_t b) {
    size_t start = b * BLOCK_SIZE;
    size_t end = std::min(start + BLOCK_SIZE, n);

    T offset = block_sum[b];
    for (size_t i = start; i < end; i++) {
        A[i] += offset;
    }
});

delete[] block_sum;
return total;
}

```

Figure 2: Second part of the implementation, where block offsets are computed and added back to each local scan result.

3 Analysis and Optimization

The total work of the algorithm is $O(n)$ because every array element is processed a constant number of times. The implementation uses two parallel passes over the full input array and one smaller sequential pass over the block sums. If the block size is B , then the number of blocks is about n/B , so the sequential offset step is much smaller than scanning all n elements directly. Therefore, the overall cost is dominated by the parallel passes, which makes the algorithm efficient for large inputs.

The main optimization is granularity control. Instead of creating one task per element, the algorithm creates one task per block. This reduces parallel overhead and gives each task enough sequential work to be useful. I also only allocate one extra array, `block_sum`, instead of allocating a large temporary array for all elements, which helps reduce memory overhead and improves performance.

4 Experimental Results

I tested the implementation using the provided executable. The parallel and sequential exclusive sums matched in all runs, which confirms correctness. I first tested the default execution without manually exporting a thread count. The average parallel running time was 0.448878 seconds, which is below the 0.5 second full-credit threshold. This result demonstrates that the implementation meets both correctness and performance requirements.

```
[twan012@xe-15 hw2-scan-HugoWan0504-1]$ make
g++ -std=c++17 -Wall -pthread -O3 -mcx16 -march=native scan.cpp -o scan
[twan012@xe-15 hw2-scan-HugoWan0504-1]$ ./scan
***** Parallel Results: *****
Exclusive sum (Parallel): 499999999500000000
Warmup round running time: 0.440613
Round 1 running time: 0.458932
Round 2 running time: 0.4461
Round 3 running time: 0.441602

***** Sequential Results: *****
Exclusive sum (Sequential): 499999999500000000
Warmup round running time (Sequential): 1.08261
Round 1 running time (sequential): 1.18701
Round 2 running time (sequential): 1.34939
Round 3 running time (sequential): 1.01169

***** Summary: *****
Average sequential running time: 1.1827
Average parallel running time: 0.448878
```

Figure 3: Default run of `./scan`, showing matching exclusive sums and an average parallel time of 0.448878 seconds.

I also tested different values of `PARLAY_NUM_THREADS`. The one-thread case is slower because it still pays the overhead of the blocked parallel algorithm without gaining parallel execution. As the thread count increases, the parallel time improves overall, with better utilization of available CPU cores. However, the speedup is not perfectly linear due to factors such as memory bandwidth limitations and scheduling overhead. The best observed time was 0.433773 seconds using 32 threads.

Thread Setting	Avg. Sequential Time (s)	Avg. Parallel Time (s)
Default	1.1827	0.448878
1 thread	0.818392	1.5009
2 threads	0.826995	0.791462
4 threads	0.828206	0.532342
8 threads	0.763593	0.472424
16 threads	0.935483	0.451612
32 threads	1.01709	0.433773

Table 1: Running time comparison for different thread settings.

```
[twan012@xe-15 hw2-scan-HugoWan0504-1]$ export PARLAY_NUM_THREADS=1
[twan012@xe-15 hw2-scan-HugoWan0504-1]$ ./scan
***** Parallel Results: *****
Exclusive sum (Parallel): 499999999500000000
Warmup round running time: 1.49663
Round 1 running time: 1.50298
Round 2 running time: 1.50364
Round 3 running time: 1.49609

***** Sequential Results: *****
Exclusive sum (Sequential): 499999999500000000
Warmup round running time (Sequential): 0.836634
Round 1 running time (sequential): 0.819115
Round 2 running time (sequential): 0.818581
Round 3 running time (sequential): 0.81748

***** Summary: *****
Average sequential running time: 0.818392
Average parallel running time: 1.5009
```

Figure 4: Runtime result with `PARLAY_NUM_THREADS=1`.

```

[twan012@xe-15 hw2-scan-Hugowan0504-1]$ export PARLAY_NUM_THREADS=2
[twan012@xe-15 hw2-scan-Hugowan0504-1]$ ./scan
***** Parallel Results: *****
Exclusive sum (Parallel): 499999999500000000
Warmup round running time: 0.791402
Round 1 running time: 0.792483
Round 2 running time: 0.790565
Round 3 running time: 0.791337

***** Sequential Results: *****
Exclusive sum (Sequential): 499999999500000000
Warmup round running time (Sequential): 0.853407
Round 1 running time (sequential): 0.827829
Round 2 running time (sequential): 0.824995
Round 3 running time (sequential): 0.828161

***** Summary: *****
Average sequential running time: 0.826995
Average parallel running time: 0.791462

```

Figure 5: Runtime result with PARLAY_NUM_THREADS=2.

```

[twan012@xe-15 hw2-scan-Hugowan0504-1]$ export PARLAY_NUM_THREADS=4
[twan012@xe-15 hw2-scan-Hugowan0504-1]$ ./scan
***** Parallel Results: *****
Exclusive sum (Parallel): 499999999500000000
Warmup round running time: 0.511491
Round 1 running time: 0.543894
Round 2 running time: 0.540345
Round 3 running time: 0.512788

***** Sequential Results: *****
Exclusive sum (Sequential): 499999999500000000
Warmup round running time (Sequential): 1.21456
Round 1 running time (sequential): 0.828784
Round 2 running time (sequential): 0.826609
Round 3 running time (sequential): 0.829225

***** Summary: *****
Average sequential running time: 0.828206
Average parallel running time: 0.532342

```

Figure 6: Runtime result with PARLAY_NUM_THREADS=4.

```

[twan012@xe-15 hw2-scan-HugoWan0504-1]$ export PARLAY_NUM_THREADS=8
[twan012@xe-15 hw2-scan-HugoWan0504-1]$ ./scan
***** Parallel Results: *****
Exclusive sum (Parallel): 499999999500000000
Warmup round running time: 0.496659
Round 1 running time: 0.460113
Round 2 running time: 0.488087
Round 3 running time: 0.469072

***** Sequential Results: *****
Exclusive sum (Sequential): 499999999500000000
Warmup round running time (Sequential): 1.79992
Round 1 running time (sequential): 0.765658
Round 2 running time (sequential): 0.762982
Round 3 running time (sequential): 0.76214

***** Summary: *****
Average sequential running time: 0.763593
Average parallel running time: 0.472424

```

Figure 7: Runtime result with PARLAY_NUM_THREADS=8.

```

[twan012@xe-15 hw2-scan-HugoWan0504-1]$ export PARLAY_NUM_THREADS=16
[twan012@xe-15 hw2-scan-HugoWan0504-1]$ ./scan
***** Parallel Results: *****
Exclusive sum (Parallel): 499999999500000000
Warmup round running time: 0.460097
Round 1 running time: 0.436619
Round 2 running time: 0.431909
Round 3 running time: 0.486307

***** Sequential Results: *****
Exclusive sum (Sequential): 499999999500000000
Warmup round running time (Sequential): 1.34197
Round 1 running time (sequential): 1.09729
Round 2 running time (sequential): 0.843832
Round 3 running time (sequential): 0.865329

***** Summary: *****
Average sequential running time: 0.935483
Average parallel running time: 0.451612

```

Figure 8: Runtime result with PARLAY_NUM_THREADS=16.

```

[twan012@xe-15 hw2-scan-HugoWan0504-1]$ export PARLAY_NUM_THREADS=32
[twan012@xe-15 hw2-scan-HugoWan0504-1]$ ./scan
***** Parallel Results: *****
Exclusive sum (Parallel): 499999999500000000
Warmup round running time: 0.433931
Round 1 running time: 0.428278
Round 2 running time: 0.435414
Round 3 running time: 0.437627

***** Sequential Results: *****
Exclusive sum (Sequential): 499999999500000000
Warmup round running time (Sequential): 1.09026
Round 1 running time (sequential): 1.33693
Round 2 running time (sequential): 0.843173
Round 3 running time (sequential): 0.871168

***** Summary: *****
Average sequential running time: 1.01709
Average parallel running time: 0.433773

```

Figure 9: Runtime result with PARLAY_NUM_THREADS=32, which produced the best measured time.

5 Conclusion

This assignment implemented an in-place blocked parallel exclusive scan. The algorithm uses parallelism at the block level, which makes the implementation simple while still giving good performance. The final output matched the sequential version in every test, and the best running time was 0.433773 seconds. Since both the default run and the best thread-count run are below 0.5 seconds, the implementation satisfies the main performance target for the assignment.

References

- Professor Yihan Sun, CS214 Coding Assignment HW2 handout.
- Professor Yihan Sun, CS214 lecture slides on reduce, work/span analysis, fork-join parallelism, and prefix sum.
- ChatGPT was used to help explain the assignment, debug the workflow, and polish the report wording.
<https://chatgpt.com/share/69eece53-0c98-8325-a9e1-673d440bf65e>